A PROJECT REPORT ON

NEXUS POS

An Offline-First Desktop Point of Sale and Intelligent Inventory Management System

For

RAJE RAMRAO MAHAVIDYALAYA, JATH

★ SHIVAJI ★
UNIVERSITY
KOLHAPUR
ज्ञानमेवामृतम्

Submitted to

SHIVAJI UNIVERSITY, KOLHAPUR

In Partial fulfillment for the Degree of

BACHELOR OF COMPUTER APPLICATION (SEM-VI)

By:

Mr. Rihan Mulla

Exam Seat No: _____

Under the guidance of:

Prof. [Guide Name]

(M.C.A)

SWAMI
VIVEKANAND
SANSTHA

Through,

The Principal,

RAJE RAMRAO MAHAVIDYALAYA, JATH

In the Academic Year: 2026 - 2027

SWAMI VIVEKANAND SANSTHA

SHRI SWAMI VIVEKANAND SHIKSHAN SANSTHA, KOLHAPUR'S

RAJE RAMRAO MAHAVIDYALAYA, JATH

DEPARTMENT OF B.C.A.

CERTIFICATE

This is to certify that the project report entitled **"NEXUS POS: An Offline-First Desktop Point of Sale and Intelligent Inventory Management System"** is a bonafide record of project work, carried out in this college by **Mr. Rihan Mulla** in partial fulfillment of the award of **Bachelor of Computer Application-III (Sem–VI)** as laid down by **Shivaji University, Kolhapur**.

This report presents their sincere work carried out under my guidance and supervision during the academic year 2026 - 2027.

Place: Jath

Date: _____ / _____ / 2026

Prof. [Guide Name]	**Mr. Mali P.D**
(M.C.A)	HOD
Project Guide	Dept. of B.C.A.

Examiners:

1) Internal Examiner: _____

2) External Examiner: _____

DECLARATION

To,
The Registrar,
Shivaji University,
Kolhapur.

Respected Sir,

We the undersigned, **Mr. Rihan Mulla**, hereby declare that the project report entitled **"NEXUS POS: An Offline-First Desktop Point of Sale and Intelligent Inventory Management System"** submitted by us under the guidance of **Prof. [Guide Name] (M.C.A)** is our original work.

We have not reproduced any information from any report submitted to Shivaji University, Kolhapur this year or in any of the previous academic years. We understand that any such copying is liable to punishment in a way that University Authorities deem fit.

Place: Jath

Date: _____ / _____ / 2026

Exam Seat No.	Name of Member	Signature
_____	Mr. Rihan Mulla	

ACKNOWLEDGEMENT

We express a very proud and greatest moment of our lives in putting forward this project entitled **"NEXUS POS"**. With the submission of this project, we would like to express our heartfelt gratitude to the individuals who have provided their valuable time, technical insights, and inspiration to accomplish this work.

At the outset, we are glad to bow with profound gratitude before our project guide **Prof. [Guide Name] (M.C.A)** for his inspiring guidance, constant encouragement, and technical directions throughout the development lifecycle of this project.

We convey our sincere thanks to **Mr. Mali P.D (Head of Department of B.C.A.)** and all respected faculty members and lab staff of Raje Ramrao Mahavidyalaya, Jath, for their continuous support and for providing computing infrastructure.

We also express our sincere respect to the Principal, **Raje Ramrao Mahavidyalaya, Jath**, for granting permission and facilities. Lastly, we thank our parents, friends, and well-wishers who stood by us and supported the progress of this project.

Place: Jath

Date: _____ / _____ / 2026

Exam Seat No.	Name of Member	Signature
_____	Mr. Rihan Mulla	

INDEX / TABLE OF CONTENTS

Sr. No.	Chapter Name & Sub-Topics	Page No.
1	**System Analysis** • 1.1 Background of the Problem • 1.2 Existing System and its Limitations • 1.3 Objective of the Project • 1.4 Scope of the Project	1
2	**System Study / Proposed System** • 2.1 Software Development Model (Agile Prototyping Model) • 2.2 Feasibility Study (Technical, Operational, Economic)	3
3	**Requirement Analysis** • 3.1 Functional Requirements • 3.2 Non-Functional Requirements • 3.3 Hardware & Software Requirements	5
4	**System Design** • 4.1 System Architecture Diagram / DFD (Level 0, Level 1, Level 2) • 4.2 Database Design (ER Diagram & Data Dictionary)	7
5	**Implementation** • 5.1 Module-wise Screenshots of Input & Output Design • 5.2 Reports (Standard A4 & 80mm Thermal Receipt Generation) • 5.3 Code Snippets (scrypt Security, Barcode Scanner, SQLite Transactions)	11
6	**Testing** • 6.1 Testing Methodology (Automated Vitest & Manual Test Cases)	16
7	**Conclusion & Future Scope** • 7.1 Summary of Work • 7.2 Limitations • 7.3 Learning Outcomes • 7.4 Possible Enhancements	18
8	**Bibliography & References**	20

CHAPTER 1: SYSTEM ANALYSIS

1.1 Background of the Problem

In physical retail stores (such as grocery supermarkets, electrical shops, clothing boutiques, and independent retail outlets in regional towns like Jath and across Maharashtra), billing and inventory tracking form the core daily operations. Historically, storekeepers relied on handwritten ledger books or basic disconnected desktop spreadsheets.

While modern cloud-based Software-as-a-Service (SaaS) POS platforms have emerged, they require uninterrupted high-speed internet connections and recurring monthly subscriptions. In semi-urban and rural retail hubs, internet connectivity frequently fluctuates or drops. When a cloud POS disconnects, the cashier cannot scan barcodes, generate bills, or calculate balances, leading to long customer queues, transaction abandonment, and severe financial losses.

1.2 Existing System and Its Limitations

The existing systems used by small merchants fall into two categories:

1. **Manual Paper Ledgers / Excel Sheets:**
    - Prone to manual mathematical errors in tax, discount, and change calculations.
    - No automated stock decrement upon billing; inventory counts require time-consuming manual audits.
    - Zero customer history tracking or loyalty rewards.
2. **Cloud-Based POS Systems (e.g., Shopify POS, Square):**
    - **Internet Vulnerability:** Complete halt of retail operations during network downtime.
    - **Prohibitive Recurring Costs:** Recurring monthly charges ($89+/month) plus transaction percentage fees.
    - **Data Privacy Risks:** Store financial records and customer phone numbers are hosted on remote third-party cloud servers.

1.3 Objective of the Project

- To build a 100% offline-first desktop application with zero internet reliance.
- To provide sub-10 millisecond billing response times using local SQLite embedded storage.
- To enforce Role-Based Access Control (RBAC) separating Admin settings from Staff checkout actions.
- To protect user credentials using memory-hard cryptographic password hashing (`scrypt`).
- To integrate hardware USB barcode scanners via keyboard emulation event listeners.
- To support dual-format invoice compilation: standard A4 documents and 80mm POS thermal receipt rolls.
- To automate inventory decrement on checkout and restock restoration on refund.

1.4 Scope of the Project

The scope of **Nexus POS** covers single-terminal physical retail establishments, including department stores, pharmacies, hardware outlets, and apparel shops. It encompasses product catalog management, barcode-driven billing, customer CRM with loyalty points, daily expense auditing, graphical revenue reporting, and local database disaster backup.

CHAPTER 2: SYSTEM STUDY / PROPOSED SYSTEM

2.1 Software Development Model

The project was developed using the **Agile Iterative Prototyping Model**. This software engineering methodology allowed rapid phase-by-phase development, continuous refinement, and rigorous verification:

- **Phase 1 (Requirements & Architecture):** Decoupled Electron multi-process design and schema creation.
- **Phase 2 (Core Inventory & POS Terminal):** Rapid UI wireframing with React and Vite.
- **Phase 3 (Billing & Dual PDF Engine):** Integrated `jsPDF` for A4 invoices and 80mm thermal rolls.
- **Phase 4 (Enterprise Security & CRM):** Implemented `scrypt` cryptographic hashing and customer loyalty points.
- **Phase 5 (Financial Auditing & Expenses):** Built daily expense tracking and net profit math.
- **Phase 6 (Automated QA & Packaging):** Wrote automated Vitest unit test suites and built Windows `.exe` installers.

2.2 Feasibility Study

A three-dimensional feasibility analysis was conducted:

Feasibility Dimension	Evaluation & Findings
Technical Feasibility	Built on Electron v39.x, React 19, and SQLite3. These technologies run natively on standard commodity PC hardware without needing expensive server configurations or high-end graphics processing units.
Operational Feasibility	The UI is built with high-contrast, modern glassmorphism dark aesthetics and intuitive ergonomics. Cashiers with basic computer literacy can scan barcodes and issue receipts with zero training.

Feasibility Dimension	Evaluation & Findings
Economic Feasibility	The software uses 100% open-source libraries and embedded SQLite. It imposes ₹0 recurring monthly server costs, zero cloud subscription fees, and eliminates recurring licensing charges.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Functional Requirements

- **FR-1 (Authentication):** Login screen with RBAC distinguishing Admin and Staff privileges.
- **FR-2 (Product Catalog):** Full CRUD capabilities for products (SKU, Name, Price, Stock).
- **FR-3 (POS Checkout):** Fast cart additions, item count toggles, and barcode scan interception.
- **FR-4 (Split-Payment & Dues):** Support Cash, Card, UPI, and partial payment balance calculation.
- **FR-5 (Invoicing Engine):** Client-side dynamic PDF compilation in A4 and 80mm thermal roll formats.
- **FR-6 (Refund & Restock):** Cancellation workflow restoring sold quantities back to inventory stock.
- **FR-7 (CRM & Loyalty):** Customer telephone lookup with automated +1 point per unit spent.
- **FR-8 (Expense Ledger):** Recording daily operational expenses (rent, electricity, tea, supplies).
- **FR-9 (Disaster Recovery):** Single-click local backup of the raw SQLite `.db` file.

3.2 Non-Functional Requirements

- **Performance:** Sub-10 millisecond database response times for instantaneous barcode lookups.
- **Availability:** 100% uptime with zero internet dependency.
- **Security:** Credential protection via salted `scrypt` hashes with timing-safe validation.
- **Data Integrity:** Full ACID compliance guaranteed by SQLite transactions.

3.3 Hardware & Software Requirements

Component	Development Environment	Client / Deployment Environment
Operating System	Windows 11 (64-bit)	Windows 10 / 11, macOS, or Linux
Processor	Intel Core i5 (11th Gen)	Dual Core x86_64 CPU (1.8 GHz+)
RAM	8 GB / 16 GB DDR4	4 GB DDR4 Minimum
Hard Disk	512 GB SSD	500 MB Free Disk Space
Runtime / Engine	Node.js v22.x, Electron v39.x	Bundled Standalone Electron Executable
Database	SQLite3 embedded	Embedded local `inventory_billing.db`
Peripherals	Keyboard, Mouse	USB Barcode Scanner & 80mm Thermal Printer

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture & Data Flow Diagrams

Nexus POS implements Electron's multi-process architecture. The **Main Process** runs in Node.js and executes privileged disk I/O and SQLite operations. The **Renderer Process** renders the React 19 UI inside a secure Chromium sandbox. Communication is strictly routed through an asynchronous **IPC (Inter-Process Communication)** bridge.

DFD Level 0: Context Diagram

External Entities: Administrator, Cashier/Staff, Customer.
Inputs to System: Credentials, Barcode scans, Cart items, Tendered payments, Expense records, Settings.
Outputs from System: Printed A4 / Thermal bills, Change due, Stock availability alerts, Visual revenue reports, SQLite backup file.

DFD Level 1: Process Decomposition

• **Process 1.0 (Auth & RBAC):** Queries Data Store D1 (`users`) via scrypt hash comparison.
• **Process 2.0 (Inventory Control):** Reads/Writes Data Store D2 (`products`).
• **Process 3.0 (POS & Billing):** Queries D2, writes transaction records to D3 (`invoices` & `invoice_items`), and fetches tax settings from D6.
• **Process 4.0 (CRM & Loyalty):** Writes accrued points to Data Store D4 (`customers`).
• **Process 5.0 (Expense Logging):** Writes daily cash outflows to Data Store D5 (`expenses`).
• **Process 6.0 (Executive Analytics):** Aggregates D2, D3, and D5 to compute net business performance.

4.2 Database Design (ER Diagram & Data Dictionary)

The database schema is fully normalized into seven relational tables in SQLite:

Table Name	Primary Key	Foreign Key(s)	Attributes & Description
`users`	`id`	None	`username` (UNIQUE), `password` (scrypt hash), `role` (Admin/Staff)
`products`	`id`	None	`name`, `sku` (UNIQUE), `price`, `stock`
`invoices`	`id`	None	`customer_name`, `customer_phone`, `date`, `total`, `amount_paid`, `balance_due`, `payment_method`, `status`
`invoice_items`	`id`	`invoice_id`, `product_id`	`quantity`, `price` (Snapshot at time of sale)
`customers`	`id`	None	`name`, `phone` (UNIQUE), `points`
`expenses`	`id`	None	`description`, `amount`, `date`
`settings`	`key`	None	`value` (Store name, currency symbol, tax percentage)

CHAPTER 5: IMPLEMENTATION

5.1 Module-wise Design Overview

1. Authentication & Login Screen

Provides a secure dark-mode portal. Authenticates users using cryptographic hash comparisons. Restricts administrative navigation links if the user role is 'Staff'.

2. POS Terminal & Split-Screen Checkout

Features dual panels: the left catalog displays product cards with live stock counts, and the right panel displays the real-time calculated cart, tax calculations, payment mode buttons, and print options.

3. Inventory Management Screen

Enables adding, updating, searching, and deleting catalog items. Visual red badges alert managers to items whose stock has reached ≤ 5 units.

4. Analytics & Executive Dashboard

Displays high-level KPI cards (Total Sales, Total Products, Low Stock Count, Today's Profit) and a clean weekly revenue performance graph.

5.2 Document & Report Generation

Nexus POS dynamically compiles invoices on the client using `jsPDF` and `jsPDF-AutoTable`. Two layout formats are available:

1. **Standard A4 Tax Invoice:** Clean full-page document including store metadata, invoice numbers, customer phone numbers, itemized tables, subtotal, tax breakdown,

and signature blocks.

2. **80mm Thermal Receipt:** Formatted specifically for POS thermal rolls with condensed typography, dashed dividers, and receipt footers.

5.3 Important Code Snippets

1. Salted Cryptographic Key Derivation (`scrypt` in Node.js):

```
const crypto = require('crypto')

// Secure User Registration
const salt = crypto.randomBytes(16).toString('hex')
const key = crypto.scryptSync(password, salt, 64).toString('hex')
const passwordHash = `${salt}:${key}`

// Timing-Safe Authentication
const [salt, key] = user.password.split(':')
const keyBuffer = Buffer.from(key, 'hex')
const derivedKey = crypto.scryptSync(enteredPassword, salt, 64)
const isValid = crypto.timingSafeEqual(keyBuffer, derivedKey)
```

2. Barcode Keystroke Listener in React (`Billing.jsx`):

```
useEffect(() => {
  const handleKeyDown = (e) => {
    if (e.key === 'Enter' && barcodeInput.length > 2) {
      const product = products.find(p => p.sku === barcodeInput)
      if (product) addToCart(product)
      setBarcodeInput('')
    } else if (e.key.length === 1) {
      setBarcodeInput(prev => prev + e.key)
      setTimeout(() => setBarcodeInput(''), 500)
    }
  }
  window.addEventListener('keydown', handleKeyDown)
  return () => window.removeEventListener('keydown', handleKeyDown)
}, [barcodeInput, products])
```

CHAPTER 6: TESTING

6.1 Testing Methodology

Software quality was validated through both **Automated Unit Testing** and **Manual Functional Verification**:

- **Automated Test Framework:** Configured with **Vitest v5.x**, **React Testing Library**, and **JSDOM**. Virtual IPC bridges were mocked to test React component lifecycles, user greetings, and cart state rendering in isolation.
- **Manual Black-Box Testing:** Tested real-world checkout workflows, barcode scanning sequences, and database backup routines.

6.2 Test Cases & Execution Results

Test ID	Scenario Under Test	Test Input	Expected Outcome	Result
TC-01	Admin Authentication	admin / admin123	Login successful; route to Dashboard	**PASS**
TC-02	Staff RBAC Security	staff1 / staff123	User Management navigation link hidden	**PASS**
TC-03	Barcode Scan Checkout	SKU "890123" + Enter	Item added to active cart automatically	**PASS**
TC-04	Inventory Auto-Deduct	Complete sale of 2 units	Stock decremented by 2 in SQLite	**PASS**
TC-05	Invoice Refund / Restock	Click "Refund" on invoice	Status marked Refunded; stock restored	**PASS**
TC-06	Loyalty Points Crediting	₹500 bill with customer phone	Customer points incremented by +500	**PASS**
TC-07	Database Local Backup	Trigger Backup in Settings	Generates identical copy of SQLite `.db` file	**PASS**

Test ID	Scenario Under Test	Test Input	Expected Outcome	Result
TC-08	Automated Vitest Suite	`npm run test`	All virtual component tests pass	**PASS**

CHAPTER 7: CONCLUSION & FUTURE SCOPE

7.1 Summary of Work

Nexus POS demonstrates the viability of modern desktop retail architectures. By integrating React 19 and Vite with Electron and SQLite, the platform provides the visual fluidity of modern web applications while delivering the speed, data privacy, and zero-latency reliability of local embedded databases. It satisfies all functional requirements set forth by physical retail merchants.

7.2 Limitations

- **Single-Terminal Scope:** Operates locally on one PC; cannot yet sync across multiple cash registers simultaneously in large supermarkets.
- **Direct Hardware Serial Hooks:** ESC/POS cash drawer triggers currently rely on standard printer drivers rather than raw serial communication.

7.3 Learning Outcomes

- Mastered Electron's decoupled multi-process architecture and secure IPC bridges using `contextBridge`.
- Gained deep practical expertise in salted cryptographic key derivation algorithms (`scrypt`) and timing-safe equality checks.
- Implemented hardware keyboard event interception in React for rapid barcode input.
- Learned automated frontend unit testing using Vitest, JSDOM, and React Testing Library.

7.4 Possible Enhancements

- **LAN Multi-Counter Sync:** Adding local WebSocket synchronization to connect multiple cashier terminals to a master register node.
- **Hardware ESC/POS Drivers:** Direct binary communication to trigger automatic physical cash drawer kickouts and thermal paper cutters.

- **Optional Encrypted Cloud Mirroring:** Scheduled end-to-end encrypted backup uploads to Google Drive or AWS S3.

CHAPTER 8: BIBLIOGRAPHY & REFERENCES

1. **Electron Documentation:** "Electron Application Architecture and Security Best Practices." *https://www.electronjs.org/docs/*
2. **React Core Team:** "React 19 Documentation & Component Paradigms." *https://react.dev/*
3. **D. Richard Hipp:** "SQLite Database System Architecture & ACID Compliance." *https://www.sqlite.org/*
4. **Colin Percival:** "Stronger Key Derivation via Sequential Memory-Hard Functions (scrypt)." *BSDCAN Proceedings, 2009.*
5. **Vitest Testing Framework:** "Blazing Fast Unit Test Framework for Vite." *https://vitest.dev/*
6. **James Parrott (jsPDF):** "Client-side HTML5 and JavaScript PDF Generation." *https://github.com/parallax/jsPDF*
7. **Pressman, Roger S.:** *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill Education.
8. **Silberschatz, Korth, Sudarshan:** *Database System Concepts*, 7th Edition, McGraw-Hill.